Tuplos e Abstracção de Dados

Fundamentos de Programação

Bernardo Toninho
IST and INESC-ID

Fundamentos de Programação
2026–2027

Baseado em slides de Luís Caires

Outline

1. Da Função ao Valor Composto
2. Representar com Tuplos
3. Interface e Representação
4. Operações sobre Rectângulos
5. Aprofundamento
6. Síntese

Da Aula 6 para a Aula 7

Na Aula 6

- encapsulámos cálculos em funções;
- definimos parâmetros e resultados;
- separámos responsabilidades;
- testámos cada cálculo isoladamente.

Nesta aula

- agrupamos vários valores relacionados;
- damos uma representação a pontos e rectângulos;
- escondemos essa representação atrás de funções;
- preservamos invariantes dos dados.

Um Problema com Dados Compostos

Queremos escrever funções para responder a perguntas como:

- onde está um ponto no plano?
- quais são os limites de um rectângulo?
- como deslocar um rectângulo?
- um ponto pertence ao rectângulo?

Nova necessidade

Uma coordenada isolada é um inteiro, mas um ponto e um rectângulo precisam de agrupar várias coordenadas num único valor.

Outline

1. Da Função ao Valor Composto
2. Representar com Tuplos
3. Interface e Representação
4. Operações sobre Rectângulos
5. Aprofundamento
6. Síntese

Um Ponto como um Único Valor

Um ponto é representado por um par ordenado de coordenadas:

```
1 Point = tuple[int, int]
2
3 p: Point = (3, 4)
4 q: Point = (-2, 5)
```

- A posição tem significado: a primeira componente é x e a segunda é y.
- O par completo é um só valor e pode ser passado a uma função.
- A anotação `Point` documenta a representação escolhida.

Tuplos em Python

```
1 p = (3, 4)
2
3 print(p[0])   # 3
4 print(p[1])   # 4
5 print(len(p)) # 2
```

- Um tuplo é uma sequência ordenada com um número fixo de componentes.
- Os índices começam em `0`.
- Os tuplos são imutáveis: não podemos substituir uma componente no lugar.

Consequência

Para representar um ponto diferente, construímos outro tuplo.

Aceder por Índice

```
1 p = (3, 4)
2 x = p[0]
3 y = p[1]
4
5 p[0] = 10  # TypeError
```

- A leitura por índice é permitida.
- A afectação a uma posição não é permitida.
- O significado de cada índice pertence ao contrato da representação.

Desempacotar um Tuplo

Podemos dar nomes às componentes numa única afectação:

```
1 p = (3, 4)
2 x, y = p
3
4 r = (8, 2, 1, 7)
5 topo, esquerda, base, direita = r
```

Leitura

O número de variáveis deve coincidir com o número de componentes do tuplo. Os nomes tornam explícito o significado de cada posição.

Previsão: Que Valores São Produzidos?

Sem executar o programa, determine o que é apresentado:

```
1 p = (6, 2)
2 x, y = p
3 x = x + 3
4
5 print(p)
6 print(x, y)
```

Alterar a variável `x` altera também o tuplo `p`?

Resultado Esperado: Desempacotamento

Problema: distinguir as componentes obtidas por desempacotamento do tuplo original.

```
(6, 2)
9 2
```

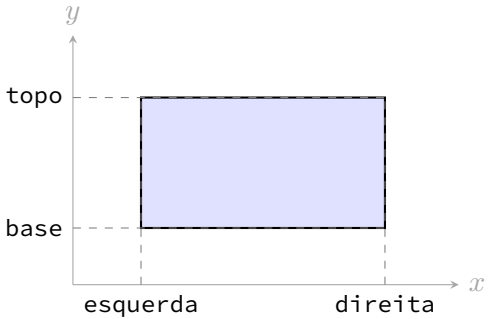
- `x` e `y` recebem os valores das duas componentes.
- A afectação posterior muda apenas a variável `x`.
- O tuplo `p` continua a ser `(6, 2)`.

Representação de um Rectângulo

Convenção

```
Rect = (topo, esquerda, base,
        direita)
```

- `topo`: maior coordenada vertical;
- `base`: menor coordenada vertical;
- `esquerda`: menor coordenada horizontal;
- `direita`: maior coordenada horizontal.



O Invariante da Representação

Nem todos os tuplos de quatro inteiros descrevem um rectângulo segundo a nossa convenção:

$$topo \geq base \qquad \text{e} \qquad direita \geq esquerda.$$

Invariante

Uma propriedade que deve ser verdadeira para todos os valores válidos de uma representação.

- O construtor deve estabelecer o invariante.
- As restantes operações devem preservá-lo.

Construir Apenas Rectângulos Válidos

```
1 Rect = tuple[int, int, int, int]
2
3 def cria_rect(topo: int, esquerda: int,
4               base: int, direita: int) -> Rect:
5     assert topo >= base
6     assert direita >= esquerda
7     return (topo, esquerda, base, direita)
```

Responsabilidade

Depois de `cria_rect` aceitar os argumentos, as outras operações podem assumir que os limites estão ordenados.

Seletores Dão Nome às Componentes

```
1  def topo(r: Rect) -> int:
2      return r[0]
3
4  def esquerda(r: Rect) -> int:
5      return r[1]
6
7  def base(r: Rect) -> int:
8      return r[2]
9
10 def direita(r: Rect) -> int:
11     return r[3]
```

`esquerda(r)` comunica a intenção; `r[1]` expõe uma posição interna.

Verificação: Rectângulos Válidos

Invariante de representação

`topo >= base` e `direita >= esquerda`

Quais destas chamadas respeitam o invariante?

- `cria_rect(8, 2, 1, 7)`
- `cria_rect(1, 2, 8, 7)`
- `cria_rect(5, 4, 5, 4)`
- `cria_rect(5, 9, 1, 3)`

Um rectângulo sem área pode ainda ser um valor válido?

Resultado Esperado: Rectângulos Válidos

Problema: verificar simultaneamente a ordem vertical e horizontal dos limites.

- `(8, 2, 1, 7)` é válido.
- `(1, 2, 8, 7)` viola `topo >= base`.
- `(5, 4, 5, 4)` é válido, mas não tem área.
- `(5, 9, 1, 3)` viola `direita >= esquerda`.

Outline

1. Da Função ao Valor Composto
2. Representar com Tuplos
3. Interface e Representação
4. Operações sobre Rectângulos
5. Aprofundamento
6. Síntese

Representação Não é Interface

Representação: como o valor é guardado, por exemplo `(x, y)`.

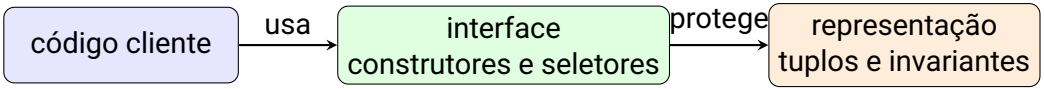
Interface: operações disponíveis para construir, observar e transformar o valor.

Cliente: código que usa essas operações sem conhecer os detalhes internos.

Objetivo

Conseguir mudar a representação sem reescrever todo o código cliente.

Anatomia de um Tipo Abstracto de Dados



- O cliente depende da interface.
- Só a implementação da interface conhece a representação.
- O conjunto forma um tipo abstracto de dados (TAD).

Interface Mínima para Pontos

```python
Point = tuple[int, int]

def cria_ponto(x: int, y: int) -> Point:
    return (x, y)

def abcissa(p: Point) -> int:
    return p[0]

def ordenada(p: Point) -> int:
    return p[1]
```

O construtor e os seletores são pequenos, mas estabelecem uma fronteira clara entre clientes e representação.

Um Cliente Usa a Interface

```
1 def soma_coordenadas(p: Point) -> int:
2     return abcissa(p) + ordenada(p)
3
4 p = cria_ponto(3, 4)
5 print(soma_coordenadas(p))  # 7
```

- O cliente sabe que existe um ponto com duas coordenadas observáveis.
- Não sabe em que posições ou estrutura essas coordenadas são guardadas.

Análise: Interface ou Representação?

Quais destas linhas dependem apenas da interface pública?

```
1 p = cria_ponto(3, 4)
2 print(abcissa(p))
3 print(p[0])
4 q = cria_ponto(abcissa(p) + 1, ordenada(p))
```

Que linha teria maior probabilidade de deixar de funcionar se a representação mudasse?

Resultado Esperado: Interface ou Representação?

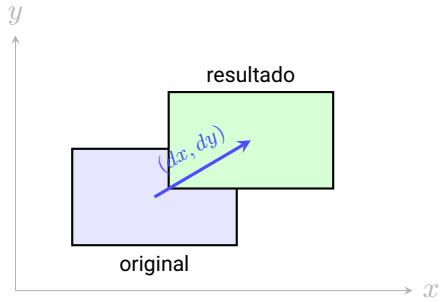
Problema: identificar o código que conhece detalhes internos do ponto.

- `cria_ponto`, `abcissa` e `ordenada` pertencem à interface.
- `p[0]` pressupõe que o ponto é um tuplo e que x ocupa a primeira posição.
- Assim, apenas o acesso directo atravessa a barreira de abstracção.

Outline

1. Da Função ao Valor Composto
2. Representar com Tuplos
3. Interface e Representação
4. Operações sobre Rectângulos
5. Aprofundamento
6. Síntese

Deslocar um Rectângulo



Contrato

`desloca_rect(r, dx, dy)` devolve um novo rectângulo obtido ao somar `dx` às coordenadas horizontais e `dy` às verticais.

O tamanho e o invariante do rectângulo devem ser preservados.

Cálculo: Actualizar os Limites

Considere:

$$r = (8, 2, 1, 7), \qquad dx = 3, \qquad dy = -2.$$

- Que limites recebem `dx`?
- Que limites recebem `dy`?
- Qual é o rectângulo resultante?
- É necessário modificar o tuplo original?

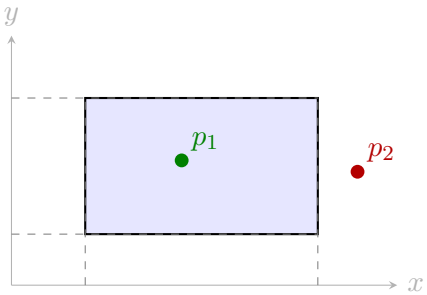
Uma Solução Possível: `desloca_rect`

Problema: transladar os quatro limites e construir um novo rectângulo.

```
1 def desloca_rect(r: Rect, dx: int, dy: int) -> Rect:
2     return cria_rect(
3         topo(r) + dy,
4         esquerda(r) + dx,
5         base(r) + dy,
6         direita(r) + dx)
```

Para o exemplo, o resultado é `(6, 5, -1, 10)`. Usar o construtor torna explícito que a operação deve devolver um valor válido.

Um Ponto Pertence ao Rectângulo?



Para pertencer ao rectângulo, o ponto tem de satisfazer ao mesmo tempo:

$$esquerda \leq x \leq direita$$

$$base \leq y \leq topo.$$

Derivar a Condição de Pertença

Para `r = cria_rect(8, 2, 1, 7)`:

- a coordenada x deve estar entre `2` e `7`;
- a coordenada y deve estar entre `1` e `8`;
- as duas condições têm de ser verdadeiras simultaneamente.

Fronteira

Como usamos `<=`, pontos sobre os lados e cantos também pertencem ao rectângulo.

Implementar `ponto_em_rect`

```
1 def ponto_em_rect(p: Point, r: Rect) -> bool:
2     x = abcissa(p)
3     y = ordenada(p)
4     return (esquerda(r) <= x <= direita(r)
5             and base(r) <= y <= topo(r))
```

- A função cliente usa apenas seletores.
- A condição traduz directamente o desenho geométrico.

Previsão: Pontos na Fronteira

Considere `r = cria_rect(8, 2, 1, 7)`. Determine os resultados:

```
1 ponto_em_rect(cria_ponto(3, 4), r)
2 ponto_em_rect(cria_ponto(9, 4), r)
3 ponto_em_rect(cria_ponto(2, 1), r)
4 ponto_em_rect(cria_ponto(7, 8), r)
```

Que chamadas testam especificamente a política escolhida para a fronteira?

Resultado Esperado: Pontos na Fronteira

Problema: aplicar simultaneamente os limites horizontal e vertical.

```
True
False
True
True
```

As duas últimas chamadas usam cantos do rectângulo e confirmam que a fronteira está incluída.

Testar a Interface e as Operações

```
1  r = cria_rect(8, 2, 1, 7)
2  assert topo(r) == 8
3  assert esquerda(r) == 2
4
5  r2 = desloca_rect(r, 3, -2)
6  assert topo(r2) == 6
7  assert esquerda(r2) == 5
8  assert base(r2) == -1
9  assert direita(r2) == 10
10 assert topo(r) == 8
11
12 assert ponto_em_rect(cria_ponto(3, 4), r)
13 assert not ponto_em_rect(cria_ponto(9, 4), r)
```

Os testes verificam resultados, preservação do original e casos interiores e exteriores.

Do Problema à Abstracção

1. Identificar os valores que formam cada entidade.
2. Escolher uma representação e declarar o seu invariante.
3. Definir construtores e seletores.
4. Escrever clientes usando apenas essa interface.
5. Testar as operações sem depender dos detalhes internos.

Ideia central

O tuplo guarda as componentes; a interface dá-lhes significado e protege a escolha de representação.

Outline

1. Da Função ao Valor Composto
2. Representar com Tuplos
3. Interface e Representação
4. Operações sobre Rectângulos
5. Aprofundamento
6. Síntese

Pergunta: Uma Função Pode Devolver um Tuplo

Pretendemos dividir `total` segundos em horas, minutos e segundos.

- Que quociente e resto resultam da divisão por `3600`?
- Sobre que valor devemos depois dividir por `60`?
- Que tuplo deve ser devolvido para `3665`?

Uma Solução Possível: Decompor Segundos

Problema: devolver três resultados relacionados num único tuplo.

```
1 def decompoe_segundos(total: int) -> tuple[int, int, int]:
2     horas, resto = divmod(total, 3600)
3     minutos, segundos = divmod(resto, 60)
4     return horas, minutos, segundos
5
6 assert decompoe_segundos(3665) == (1, 1, 5)
```

Pergunta: Imutabilidade

Considere:

```
1 p = cria_ponto(3, 4)
2 p[0] = 8
```

- O que acontece na segunda linha?
- Como podemos obter um ponto com abcissa `8` e a mesma ordenada?

Resultado Esperado: Construir um Novo Valor

Problema: alterar conceptualmente um valor representado por um tuplo imutável.

```
1 p = cria_ponto(3, 4)
2 q = cria_ponto(8, ordenada(p))
```

`p[0] = 8` produz `TypeError`; a alternativa é construir `q`. O valor original permanece inalterado.

Dimensões de um Rectângulo

Usando apenas a interface:

$$largura(r) = direita(r) - esquerda(r)$$

$$altura(r) = topo(r) - base(r).$$

O invariante garante que ambas as diferenças são não negativas.

Reutilização

Área e perímetro podem ser definidos a partir destas duas funções, sem voltar a conhecer a representação.

Pergunta: Área e Perímetro

Defina os contratos de `area_rect(r)` e `perimetro_rect(r)`.

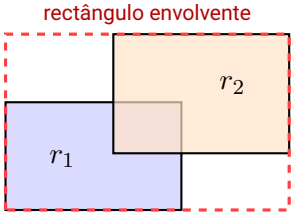
- Que funções auxiliares devem ser reutilizadas?
- Que resultados espera para `cria_rect(8, 2, 1, 7)`?
- O que acontece num rectângulo sem área?

Uma Solução Possível: Área e Perímetro

Problema: calcular medidas sem aceder directamente às posições do tuplo.

```
1  def largura(r: Rect) -> int:
2      return direita(r) - esquerda(r)
3
4  def altura(r: Rect) -> int:
5      return topo(r) - base(r)
6
7  def area_rect(r: Rect) -> int:
8      return largura(r) * altura(r)
9
10 def perimetro_rect(r: Rect) -> int:
11     return 2 * (largura(r) + altura(r))
```

União de Dois Rectângulos



Contrato

`uniao_rect(r1, r2)` devolve o menor rectângulo que contém integralmente os dois argumentos.

Pergunta: Escolher os Limites Exteriores

Para construir o rectângulo envolvente:

- o topo deve ser o maior ou o menor dos dois topos?
- a esquerda deve ser a maior ou a menor das duas esquerdas?
- que escolhas análogas fazemos para base e direita?
- como preservamos a barreira de abstracção?

Uma Solução Possível: `uniao_rect`

Problema: escolher os quatro limites exteriores dos argumentos.

```
1 def uniao_rect(r1: Rect, r2: Rect) -> Rect:
2     return cria_rect(
3         max(topo(r1), topo(r2)),
4         min(esquerda(r1), esquerda(r2)),
5         min(base(r1), base(r2)),
6         max(direita(r1), direita(r2)))
```

Todos os acessos passam pelos seletores e o resultado passa pelo construtor.

Pergunta: E se a Representação Mudar?

Suponha que passamos a guardar as coordenadas pela ordem `(y, x)`:

```
1 Point = tuple[int, int]  # agora (y, x)
```

- Que funções da interface têm de mudar?
- `soma_coordenadas` precisa de ser reescrita?
- Que aconteceria a um cliente que usasse `p[0]`?

Uma Resposta Possível: Adaptar a Interface

Problema: alterar a representação sem alterar clientes bem escritos.

```
1 def cria_ponto(x: int, y: int) -> Point:
2     return (y, x)
3
4 def abcissa(p: Point) -> int:
5     return p[1]
6
7 def ordenada(p: Point) -> int:
8     return p[0]
```

`soma_coordenadas` continua correcta porque usa apenas a interface. Um cliente que usasse `p[0]` passaria a interpretar y como x.

Outline

1. Da Função ao Valor Composto
2. Representar com Tuplos
3. Interface e Representação
4. Operações sobre Rectângulos
5. Aprofundamento
6. Síntese

Conclusão

Hoje vimos

- tuplos para agrupar valores relacionados
- invariantes, construtores e seletores
- interfaces que escondem a representação

A seguir

- listas e dicionários para coleções
- operações de procura e actualização
- registos com campos identificados